

MODULE 9 · TROUBLESHOOTING

Resource & HPA Problems

The scheduler, the kernel, and the HPA all read the same numbers

Resource failures rarely look like failures

An OOMKill restarts the container so fast the logs are empty. CPU throttling never fires an event at all. An HPA stuck at <unknown> isn't erroring — it's quietly unable to compute a number.

Four different symptoms, one family of causes: a requests/limits block, and whether the scheduler or the HPA can actually read it.

BY THE END YOU CAN

- Tell OOMKilled from CPU throttling
- Read a taint vs a nodeSelector Pending
- Trace an HPA <unknown> to its real cause

CONCEPT

One family, four symptoms

CLASS	WHEN IT HAPPENS	WHERE TO LOOK
Runtime — OOMKilled	Container started fine, then exceeded <code>limits.memory</code>	<code>describe pod</code> → Last State
Runtime — CPU throttled	Container's CPU demand exceeds <code>limits.cpu</code>	<code>top pod</code> pinned at limit — no event
Scheduling — Pending	API accepted the pod, scheduler can't place it	<code>describe pod</code> → Events → <code>FailedScheduling</code>
HPA — <code><unknown></code> targets	HPA controller can't compute <code>current/target</code>	<code>describe hpa</code> → Conditions

The HPA row isn't a different problem — it's a controller reading the same signals a Pod's scheduling and runtime already depend on.

Same over-limit, two different kernel responses

Memory is **incompressible** — exceed the limit and the kernel has no choice but to SIGKILL the container. CPU is **compressible** — exceed the limit and the CFS quota just throttles it instead.

Throttling produces no restart and no Warning event. The only signature is usage pinned flat at the limit while the app quietly slows down.

	OOMKILLED	THROTTLED
Response	SIGKILL, exit 137	CFS pause, no kill
Status	CrashLoopBackOff	Running , 0 restarts
Signal	Last State: OOMKilled	nr_throttled > 0

Same status, two different event phrases

CAUSE	EVENT PHRASE	FIX
Untolerated taint	untolerated taint(s)	Add the matching toleration
nodeSelector mismatch	didn't match Pod's node affinity/selector	Relax the selector, or label the node

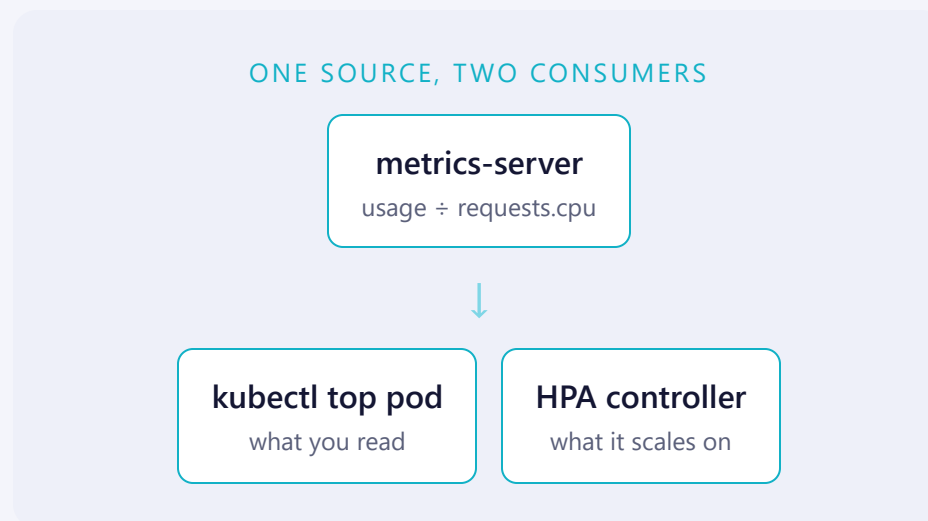
ALREADY COVERED

Lab 9.1 covered the third Pending cause — a request bigger than any node has free (Insufficient cpu / Insufficient memory). Same status, three distinct event phrases.

HPA reads the exact same signals as `kubectl top`

Everything so far was one Pod's own resources. The HPA is just another consumer of the same pipeline: live usage from **metrics-server**, divided by `requests.cpu` on the container.

Break either ingredient — no metrics-server, or no `requests.cpu` — and both `kubectl top` *and* the HPA go blind, for the same reason.



REPLICAS: 0 means it can't even find the Deployment

`scaleTargetRef.name` is just a string — a typo points the HPA at a Deployment that doesn't exist. It can't get a scale to read, so it can't scale anything.

Fix: correct `scaleTargetRef.name` to match the real Deployment.

```
$ kubectl get hpa -n training
REFERENCE                               TARGETS             REPLICAS
Deployment/php-apache-typo              <unknown>/50%      0

$ kubectl describe hpa php-apache-hpa -n training
AbleToScale  False  FailedGetScale
deployments.apps "php-apache-typo" not found
```

REPLICAS: 1 but TARGETS still <unknown>

Fix the target ref and the HPA finds the Deployment — but utilization is $\text{usage} \div \text{requests.cpu}$. No `requests.cpu` on the container means "50% of nothing": undefined.

Fix: add `resources.requests.cpu` to the target Deployment's container.

```
$ kubectl get hpa -n training
```

REFERENCE	TARGETS	REPLICAS
Deployment/php-apache	<unknown>/50%	1

```
$ kubectl describe hpa php-apache-hpa -n training
ScalingActive  False  FailedGetResourceMetric
missing request for cpu in container php-apache
```


What the status is telling you

STATUS	SIGNATURE	CAUSE
CrashLoopBackOff	Last State: OOMKilled , exit 137	Memory limit too low
Running , 0 restarts	top flat at limit, nr_throttled>0	CPU limit too low
Pending	untolerated taint(s)	Missing toleration
Pending	didn't match node affinity/selector	nodeSelector mismatch
HPA <unknown> , REPLICAS:0	FailedGetScale , "not found"	Wrong scaleTargetRef
HPA <unknown> , REPLICAS:N	FailedGetResourceMetric	Missing requests.cpu

Where people trip

- **Chasing app logs on an OOMKill.** SIGKILL fires before the process can write anything — the logs are empty by design.
- **Missing throttling entirely.** No restart, no event — if usage sits pinned at the limit, check the cgroup counter, not get pod.
- **Treating <unknown> as "still warming up."** It's a signal the controller couldn't get a value — the reason is always in Conditions.
- **Fixing the target ref and stopping there.** A correct scaleTargetRef can still show <unknown> if requests are missing — two separate fixes.

Commands to keep at hand

```
$ kubectl describe pod NAME           # Last State + Events
$ kubectl top pod NAME                 # usage vs limit – is it pinned?
$ kubectl exec NAME -- cat /sys/fs/cgroup/cpu.stat # nr_throttled, the definitive signal
$ kubectl describe node NAME           # Taints: line
$ kubectl get hpa -n NS                # TARGETS / REPLICAS at a glance
$ kubectl describe hpa NAME -n NS      # Conditions – the real reason
```

Tip: <unknown> is never a wait state. The reason lives in the HPA's Conditions, every time.

RECAP

Resource & HPA triage in one breath

- Memory over limit kills; CPU over limit only throttles
- Pending event text names taint vs selector vs capacity
- HPA reads the same requests + metrics-server as top
- <unknown> → read Conditions, don't guess

HANDS-ON

Now run Lab 9.9
`labs/9.9/`

Next: [9.10 — Node Operations](#)